

14

Local effects and mutable state

In the first chapter of this book, we introduced the concept of referential transparency, setting the premise for purely functional programming. We declared that pure functions can't mutate data in place or interact with the external world. In chapter 13, we learned that this isn't exactly true. We *can* write purely functional and compositional programs that describe interactions with the outside world. These programs are unaware that they can be interpreted with an evaluator that has an effect on the world.

In this chapter we'll develop a more mature concept of referential transparency. We'll consider the idea that effects can occur *locally* inside an expression, and that we can guarantee that no other part of the larger program can observe these effects occurring.

We'll also introduce the idea that expressions can be referentially transparent *with regard to* some programs and not others.

14.1 Purely functional mutable state

Up until this point, you may have had the impression that in purely functional programming, you're not allowed to use mutable state. But if we look carefully, there's nothing about the definitions of referential transparency and purity that disallows mutation of *local* state. Let's refer to our definitions from chapter 1:

Definition of referential transparency and purity

An expression e is referentially transparent if for all programs p all occurrences of e in p can be replaced by the result of evaluating e without affecting the meaning of p .

A function f is pure if the expression $f(x)$ is referentially transparent for all referentially transparent x .

By that definition, the following function is pure, even though it uses a while loop, an updatable var, and a mutable array.

Listing 14.1 In-place quicksort with a mutable array

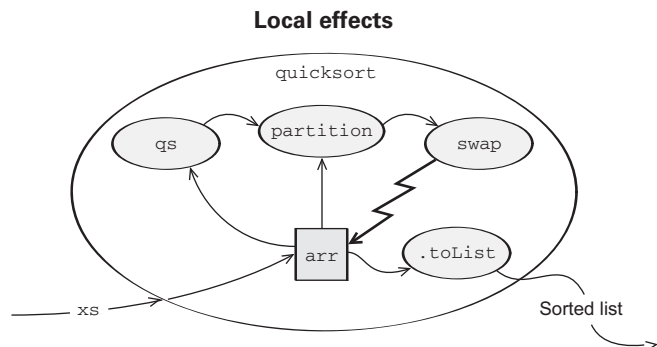
```
def quicksort(xs: List[Int]): List[Int] = if (xs.isEmpty) xs else {
  val arr = xs.toArray
  def swap(x: Int, y: Int) = {
    val tmp = arr(x)
    arr(x) = arr(y)
    arr(y) = tmp
  }
  def partition(n: Int, r: Int, pivot: Int) = {
    val pivotVal = arr(pivot)
    swap(pivot, r)
    var j = n
    for (i <- n until r) if (arr(i) < pivotVal) {
      swap(i, j)
      j += 1
    }
    swap(j, r)
    j
  }
  def qs(n: Int, r: Int): Unit = if (n < r) {
    val pi = partition(n, r, n + (n - r) / 2)
    qs(n, pi - 1)
    qs(pi + 1, r)
  }
  qs(0, arr.length - 1)
  arr.toList
}
```

← Swaps two elements in an array

← Partitions a portion of the array into elements less than and greater than pivot, respectively

← Sorts a portion of the array in place

The quicksort function sorts a list by turning it into a mutable array, sorting the array in place using the well-known quicksort algorithm, and then turning the array back into a list. It's not possible for any caller to know that the individual subexpressions inside the body of quicksort aren't referentially transparent or that the local methods swap, partition, and qs aren't pure, because at no point does any code outside the quicksort function hold a reference to the mutable array. Since all of the mutation is locally scoped, the overall function is pure. That is, for any referentially transparent expression xs of type List[Int], the expression quicksort(xs) is also referentially transparent.



A mutation that happens inside a function is not a side effect if nothing outside the function refers to the mutated object.